MSCCL: A Framework for Enhancing Mashup Service Clustering With Contrastive Learning

Qiang Hu, Haoquan Qi, Shengzhi Du, and Pengwei Wang, *Member, IEEE*

***Abstract*—Obtaining high-quality service function vectors and aggregating neighborhood features in service association graph are prevalent methods for Mashup service clustering. However, existing methods often focus on enhancing the service functional feature extraction while overlooking distinctions among different services when creating service function vectors. Additionally, neighborhood feature aggregation is typically considered within a single association graph, lacking contrast optimization of different association features. To address these challenges, we propose a novel framework, MSCCL (Mashup Service Clustering with Contrastive Learning). MSCCL consists of two core components: a service function vector generation module and a neighborhood feature aggregation module. Contrastive learning is employed to enhance vector quality and optimize feature aggregation in both modules. We present a service clustering method within MSSCL that combines techniques from BERT (Bidirectional Encoder Representations from Transformers) and GAT (Graph Attention Networks). Compared to state-of-the-art methods, this approach reduces DBI by 2.03% to 12.58%, while enhancing SC, NMI, and Purity by 2.24% to 15.47%, 3.34% to 11.39%, and 2.58% to 13.65%, respectively. Furthermore, the experiments demonstrate that the popular models for service function vector generation and neighborhood feature aggregation can all be integrated into MSSCL. After being integrated into MSSCL, the clustering performance of these models was significantly improved, highlighting the effectiveness and generalizability of MSSCL.**

***Index Terms*—Mashup service, contrastive learning, service clustering, graph attention networks (GAT), bidirectional encoder representations from transformers (BERT).**

I. Introduction

WEB service represents a category of network application modules characterized by their self-contained, modular, loosely coupled, standards-based, and high-capacity nature [1]. The Internet hosts an extensive array of Web services tailored for various applications, playing a pivotal role in software development and deployment rooted in service-oriented architecture. These Web services are deliberately designed with small functional granularity to simplify invocation and integration. Generally, a single Web service alone cannot adequately fulfill complex business requirement [2].

To tackle complex business scenarios, Mashup service has appeared as a new form of service organization. By combining multiple business-related Web services, Mashup services create value-added solutions that integrate a broader range of data resources and deliver more business functionalities [3]. In recent years, the rapid growth of Web services and Mashup services on various service registration platforms has provided users with more options, but it has also made discovering suitable services increasingly challenging. Service clustering classifies similar services into distinct groups, reducing the service search space and, consequently, the time required for service lookup. This approach enhances the efficiency of both service discovery and composition.

Service clustering is commonly performed by assessing the similarity of service function vectors generated from service descriptions. To produce high-quality service function vectors, efforts have focused on improving generation methods through model and corpus. Various models, such as TE-GSDMM [4], CWSR-K [5] and FC-LSTM [6], have been proposed. Researchers often enrich the semantics of generative corpus by expanding service descriptions, utilizing domain ontologies, or transferring information from external knowledge repositories [7], [8], [9]. In addition, structural relationships, such as tag co-occurrence and component API co-occurrence, are commonly utilized to build service association graphs. Aggregating neighborhood features from these graphs has become a widely adopted strategy for further improving the quality of service function vectors.

Numerous methods have been proposed to enhance the quality of service function vectors. However, most of these approaches focus solely on improving service functional feature extraction, overlooking the inherent functional differences among various Mashup services. Leveraging these distinctions could further refine service function vectors, leading to more precise clustering. Additionally, many existing methods confine neighborhood feature aggregation to a single association graph, failing to explore the potential enhancements that could be achieved by optimizing across multi-dimensional associations.

Received 11 October 2024; revised 26 March 2025; accepted 22 April 2025. Date of publication 29 April 2025; date of current version 12 June 2025. This work was supported by the Natural Science Foundation of China under Grant 61973180 and in part by the Key Research Program of Shandong Province (Soft Sciences) under Grant 2023RKY01009. *(Corresponding author: Pengwei Wang.)*

Qiang Hu is with the College of Information Science and Technology, Qingdao University of science and technology, Qingdao 266061, China (e-mail: huqiang200280@163.com).

Haoquan Qi is with Donghua University, Shanghai 201620, China (e-mail: 1249141@mail.dhu.edu.cn).

Shengzhi Du is with the Department of Electrical Engineering, Tshwane University of Technology, Pretoria 0183, South Africa (e-mail: dushengzhi@gmail.com).

Pengwei Wang is with Donghua University, Shanghai 201620, China (e-mail: wangpengwei@dhu.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TSC.2025.3565389, provided by the authors.

Digital Object Identifier 10.1109/TSC.2025.3565389

1939-1374 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

It is worth noting that the more similar the shared features within service function vectors and the more distinct their unique features, the more discriminative these vectors become, leading to higher-quality clustering. Contrastive learning focuses on capturing common features among instances within the same class while amplifying the differences between instances of different classes [10]. Consequently, introducing contrastive learning into the process of generating service function vectors and aggregating neighborhood features holds significant potentials for enhancing the quality of Mashup service clustering.

To this end, we propose Mashup Service Clustering with Contrastive Learning (MSSCL)—a novel framework designed to enhance service clustering performanc

- A verb-substitution-based text data augmentation method is proposed to create twin samples for service descriptions. By incorporating both original and twin descriptions into contrastive learning, this approach effectively captures functional differences across services, refines the distribution of functional features, and enhances the quality of service function vectors.
- A graph data augmentation method for aggregating neighborhood features within dual service association graphs is proposed. This method leverages contrastive learning to optimize the aggregation of service neighborhood features. By refining feature dependencies in two types of association graphs, it significantly improves the quality of neighborhood feature aggregation for Mashup services.
- Extensive experiments on public datasets demonstrate that embedding MSSCL enhances the clustering quality of widely used models. These results confirm the effectiveness of MSSCL and highlight its strong generalization capability.

The rest of the paper is organized as follows. Section II introduces related work on service clustering. The proposed framework MSCCL is elaborated in Section III. The performance of MSCCL is verified in Sections IV. Section V concludes the paper and outlines future work.

II. Related Work

The rich functionality of a Mashup service is achieved by its constituent Web services. We briefly review two branches of research closely related to our work, including Web service clustering and Mashup service clustering, and summarize the shortcomings of the existing methods.

A. Web Service Clustering

Web services are typically described by natural language. To generate service function vectors from service descriptions, topic models or neural network models are commonly employed. Xie et al. [11] expanded the feature set of service descriptions using the RGPS (Role-Goal-Process-Service) meta-model framework, extracting latent service features via the Bi-term Topic Model (BTM) and clustering them using the k-means algorithm. Agarwal et al. [12] leveraged Word2Vec and GSDMM to extract relevant features and semantic relationships from Web services, applying a genetic clustering method based on Manhattan distance, which yielded superior results compared to other service clustering methods.

Shen et al. [13] introduced a variant of Latent Dirichlet Allocation (LDA) that incorporated a probability correction factor to improve topic extraction. Their method outperformed standard LDA, other variants, and state-of-the-art topic models for service clustering. Hu et al. [14] proposed a method that combines both textual and numerical attributes to calculate the similarity of cloud manufacturing services. Experimental results demonstrate that, compared to traditional clustering methods that primarily focus on functional similarity, their approach significantly enhances clustering quality.

In recent research, integrating service associations has become a popular approach for improving clustering quality. For instance, Kang et al. [15] extracted content semantics from service descriptions and structural semantics from service relationship networks, combining the two by training a service classification model. A spectral clustering algorithm was then applied to these unified features to cluster Web services. Ping et al. [16] proposed the Document Weight and Tag Weight-LDA model to extract the functional semantics of Web services, further enhancing clustering accuracy by merging separate networks for text and tags into a unified clustered service network.

Zou et al. [17] introduced DeepWSC, a framework for Web service clustering that extracts deep semantic features from service descriptions using a recurrent convolutional neural network. It also incorporates service composability features via a signed graph convolutional network, generating integrated service function vectors for clustering. Hu et al. [4] proposed a method that combines service function similarity and collaboration similarity to cluster Web services. Their experiments demonstrated that incorporating service collaboration can significantly enhance service clustering quality.

B. Mashup Service Clustering

In the field of Mashup service clustering, Rahman et al. [18] employed a two-level topic model to cluster Mashup services. They modeled the Mashup service network by evaluating common APIs invoked by Mashup services and the tags used to categorize them. Subsequently, Mashup service topics were extracted at both content and network levels for service clustering. Ramathulasi et al. [19] proposed an enhanced LDA model called AUG-LDA, which integrates word embedding clusters. By vectorizing Mashup service descriptions through AUG-LDA, they generated high-quality service vectors, which can be effectively clustered using the K-means++ algorithm. Lu et al. [20] applied GSDMM to automatically estimate the number of topics for Mashup services. They combined word embeddings and service tag information with non-negative matrix factorization to obtain the topic features. Finally, a spectral clustering algorithm is employed to cluster Mashup services.

Cao et al. [21] designed a domain-aware Mashup service feature selection and reduction approach. They employed an extended LDA model derived from multiple data sources, including Mashup description text, Web APIs, and tags, to infer the topic probability distribution of Mashup services. These

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

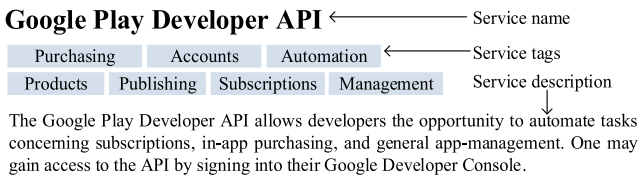


Fig. 1. An example of Web service.

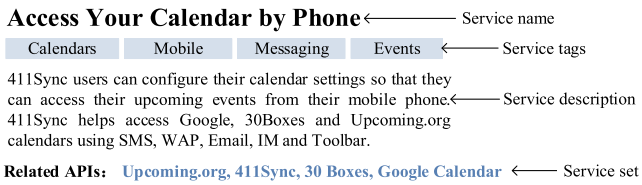


Fig. 2. An example of mashup service.

topic probability distributions served as the basis for similarity computation in Mashup service clustering. Pan et al. [22] introduced a two-mode graph to describe Mashup services, Web APIs, and their relations. They used the SimRank algorithm to quantify structural similarity between Mashup services and utilized the genetic algorithm for clustering. They also proposed a method to cluster Mashup services by integrating structural similarity and semantic similarity using fuzzy AHP (analytic hierarchy process). Structural similarity was computed from usage histories between Mashup services and Web APIs, while semantic similarity was derived from Mashup descriptions and tags using LDA [23].

C. Summary

While service functional features are key to clustering performance, integrating service associations can significantly enhance clustering quality. Most research focuses on feature extraction models or extensive corpus, but few address differences between services in generating function vectors. These distinctions can improve vector differentiation and overall clustering. Likewise, various service association graphs have been developed to improve clustering quality, but most of these approaches overlook a comparative analysis of service differences when aggregating association features.

To address this, we propose incorporating contrastive learning to emphasize distinctions in both functional and association features, improving Mashup service clustering.

III. Mashup Service Clustering With Contrastive Learning

A. Definitions

As the functionality of a Mashup service is collaboratively implemented by its constituent Web services, a Mashup service typically consists of multiple Web services. We begin by formally defining both Web services and Mashup services. Fig. 1 presents an example of a Web service, while Fig. 2 shows a Mashup service.

Definition 1. Web service: A Web service is defined as a 3-tuple $s = (n, T, d)$, where n represents the service name, T is the set of service tags, and d denotes the service description.

Definition 2. Mashup service: A Mashup service is defined as a 4-tuple $m = (n, T, d, S)$, where n is the service name, T is the set of service tags, and d refers to the service description, and S represents the set of constituent Web services.

Definition 3. Tag Sharing: Mashup services m_i and m_j are called tag sharing if there exists a service tag t, s.t., $t \in (m_i.T \cap m_j.T)$, denoted as $m_i \sim m_j$.

Definition 4. Service sharing: Mashup services m_i and m_j are called service sharing if there exists a Web service s, s.t., $s \in (m_i.S \cap m_j.S)$, denoted as $m_i \leftrightarrow m_j$.

Definition 5. Mashup service association graph oriented tag sharing: A Mashup service association graph oriented tag sharing is an undirected weighted graph T_MAG $= (V, E, W)$ if the following conditions are held:

1) $V = \{v_1, v_2, \ldots, v_n\}$ is the set of service nodes, where the node v_i represents the Mashup service m_i.
2) $E = \{e_{ij}\}$ is the set of service associated edges, and $e_{ij} = (v_i, v_j)$ means that the Mashup services m_i and m_j satisfy $m_i \sim m_j$;
3) $W = \{w_{ij}\}$ is the set of edge weights, where w_{ij} represent the number of tags shared between Mashup services m_i and m_j.

Definition 6. Mashup Service Association Graph Oriented Service Sharing: A Mashup service association graph oriented service sharing is an undirected weighted graph S_MAG $= (V, E, W)$, where the following conditions hold:

1) $V = \{v_1, v_2, \ldots, v_n\}$ is the set of service nodes, where the node v_i represents the Mashup service m_i.
2) $E = \{e_{ij}\}$ is the set of service associated edges, and $e_{ij} = (v_i, v_j)$ means that the Mashup services m_i and m_j satisfy $m_i \leftrightarrow m_j$;
3) $W = \{w_{ij}\}$ is the set of edge weights, where w_{ij} represents-the number of shared constituent Web services between Mashup service m_i and m_j.

B. Overview of MSCCL

The backbone of MSCCL is illustrated in Fig. 3, consisting of three modules: the service function vector generation module with contrastive learning (outlined by the green dashed box), the neighborhood feature aggregation module with contrastive learning (highlighted by the red dashed box), and the clustering algorithm module.

As shown in the black solid line box in Fig. 3, the input to the MSCCL framework is a set of Mashup services to be clustered. In the service function vector generation module, we design a verb synonym substitution method to create the twin samples for Mashup services. A topic model or neural network model is then used to generate text vectors from the initial service descriptions and the twin service descriptions. These vectors are paired, and service function vectors are produced through contrastive learning.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

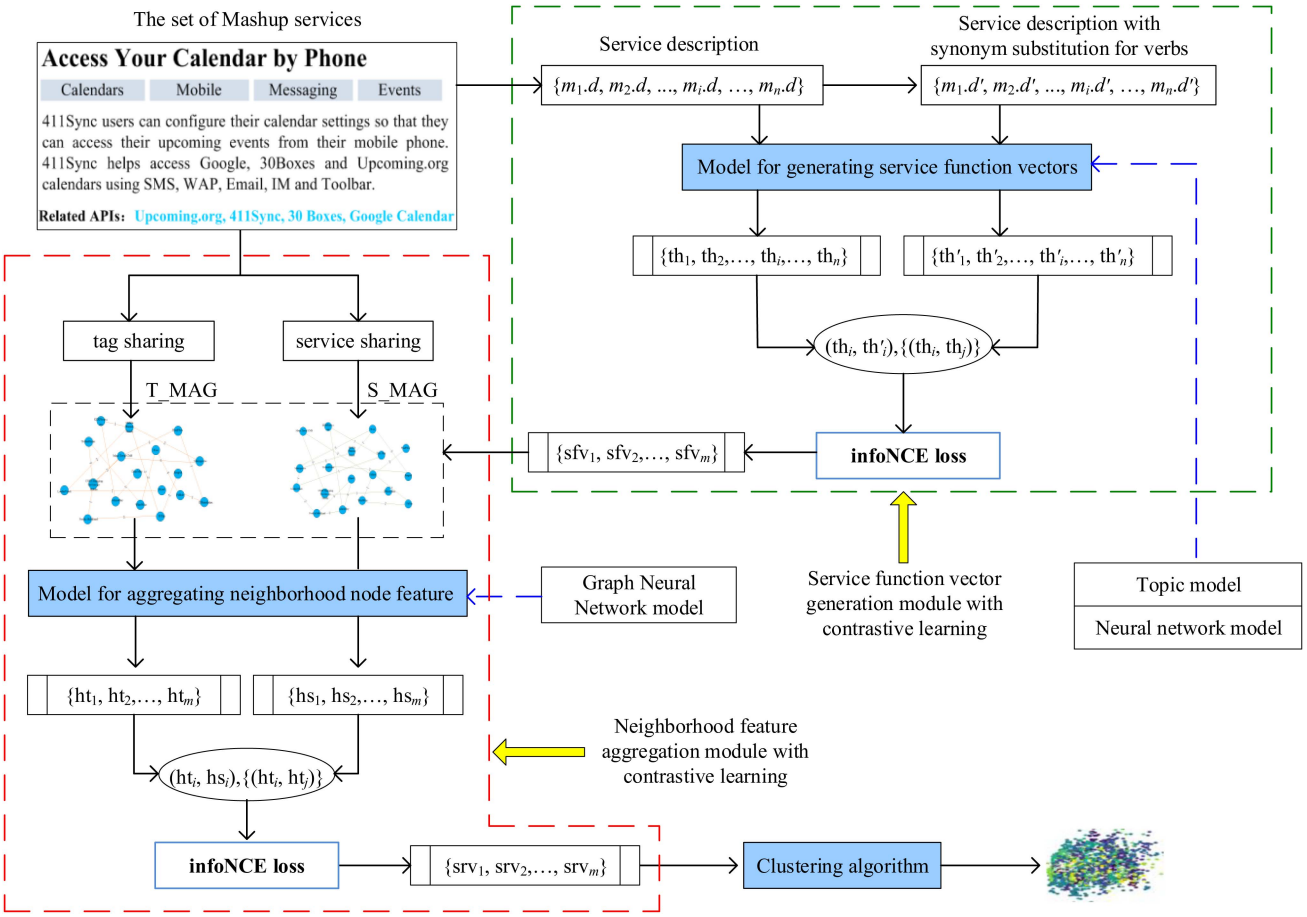


Fig. 3. Backbone of MSCCL.

In the neighborhood feature aggregation module, we construct two distinct Mashup service association graphs: the Tag-sharing oriented Mashup service Association Graph (T_MAG) and the Web service-sharing oriented Mashup service Association Graph (S_MAG). These graphs are built based on tag-sharing relationships and shared component Web services among Mashup services. The service nodes are initially embedded with service function vectors, and graph neural network models, such as GCN (Graph Convolutional Network) or GAT, are employed to aggregate neighborhood features across T_MAG and S_MAG.

The above process generates service representation vectors for Mashup services, incorporating neighborhood features from different association perspectives. These representation vectors are then utilized to construct positive and negative pairs, further refining the final service representation vectors through contrastive learning. Finally, a clustering algorithm is applied to group Mashup services based on the resulting representation vectors.

As an overarching framework designed to enhance the quality of Mashup service clustering, MSCCL is composed of three modular components, each of which can be substituted with alternative models. These interchangeable components are highlighted by the blue rectangles in Fig. 3.

C. Service Function Vector Generation Via Contrastive Learning

Data augmentation is commonly used to construct sample pairs in contrast learning [24]. Random deletion, back-translation, random synonym substitution, and random dropout stand as common data augmentation strategies in natural language processing tasks [25]. Given the brevity of Mashup service descriptions and the predominance of nouns, particularly proper nouns, the selection of nouns for deletion or substitution to construct positive instances might introduce significant semantic differences between the new generated instance and the original service description.

In Mashup service descriptions, verbs usually represent service operations. Enhancing the functional semantics of verbs can improve the quality of generated service function vectors. In light of this, we propose a data augmentation approach based on verb synonym substitution to generate twin samples from service descriptions. As illustrated in Fig. 4, the original and twin service descriptions are used to construct positive and negative pairs for contrastive learning.

The Stanford NLP tool is employed to tag the part-of-speech of words in $m_i.d$, where $m_i.d = \{w_1, w_2, \ldots, w_n\}$ represents the description text of Mashup service m_i. The set of Mashup service descriptions is denoted as $D = \{m_1.d, m_2.d, \ldots, m_n.d\}$. Suppose

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

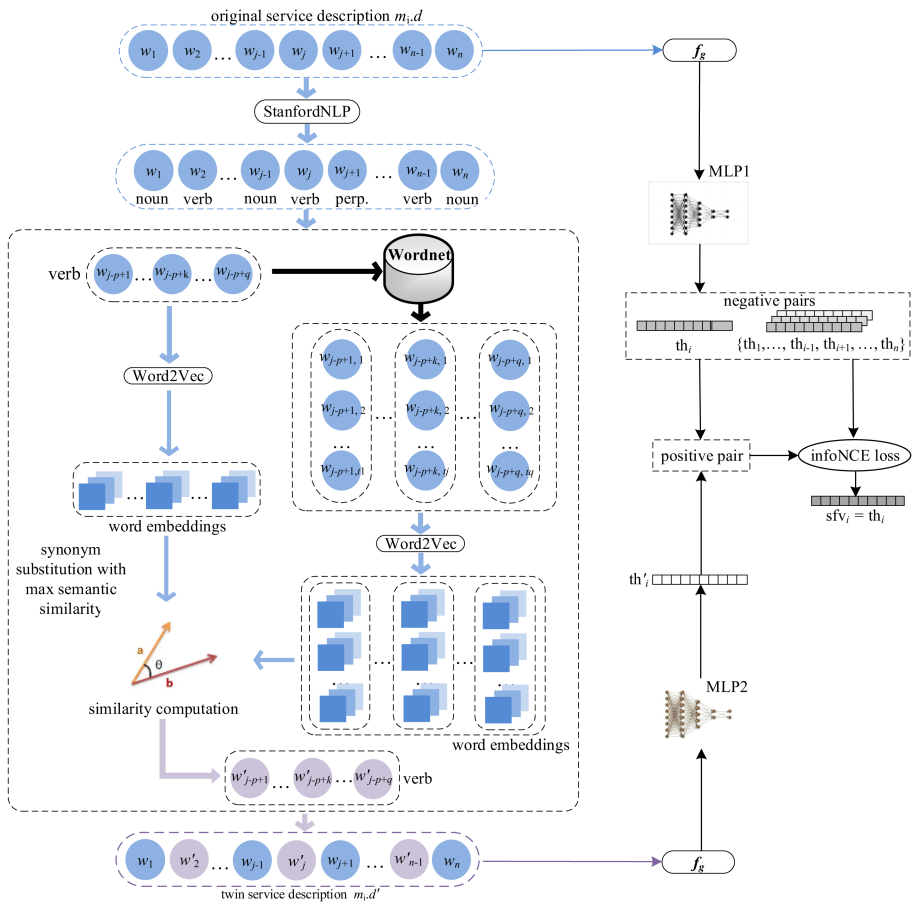


Fig. 4. A data augmentation method based on verb synonym substitution.

there are a total of q verbs in $m_i.d$, represented as $\{w_{j\text{-}p+1}, \ldots, w_{j\text{-}p+k}, \ldots, w_{j\text{-}p+q}\}$.

We can use WordNet to find synonyms for a given word. Let $S_{yn} = \{w_{j\text{-}p+k,1}, w_{j\text{-}p+k,2}, \ldots, w_{j\text{-}p+k,tj}\}$ be the synset of $w_{\text{j-p+k}}$, then the word embedding for all the words in S_{yn} and $w_{\text{j-p+k}}$ can be generated. Next, the cosine similarity between these embeddings is calculated to determine the semantic similarity between the words in S_{yn} and $w_{j\text{-}p+k}$. The word with the highest semantic similarity is selected as the synonym to replace $w_{j\text{-}p+k}$. The service description generated after replacing all verbs in $m_i.d$ with their synonyms is denoted as $m_i.d'$, referred to as the "twin sample" of $m_i.d$.

Let f_d be the model to generate service function vectors in Mashup service clustering. This model generate service function vectors for both $m_i.d$ and $m_i.d'$ respectively, denoted as:

$$\text{hd}_\text{i} = f_d\left(\text{m}_\text{i}.d\right) \tag{1}$$

$$\text{hd}_\text{i}' = \text{f}_\text{d}\left(m_i.d'\right) \tag{2}$$

hd_i and hd_i' are then passed through a feature optimization layer consisting of a fully connected layer and a sigmoid activation function, to produce the optimized vectors th_i and th_i', respectively.

$$\text{th}_\text{i} = \text{sigmoid}\left(W_{t1}\text{hd}_i + b_{t1}\right) \tag{3}$$

$$\text{th}_\text{i}' = \text{sigmoid}\left(W_{t2}\text{hd}_i' + b_{t2}\right) \tag{4}$$

Where, W_{t1} and W_{t2} are trainable weight matrices, b_{t1} and b_{t2} are bias vectors. These trainable parameters are shared across the optimization of text vectors for all service descriptions. The vectors th_i and th_i' are coupled as positive pairs for m_i. Simultaneously, the text vector th_i and the text vector th_j generated by other service descriptions in the same batch without verb synonym substitution are used to construct negative pairs.

The following infoNCE function is used as the training loss:

$$\mathcal{L}_{fv} = -\log \frac{\text{e}^{\text{sim}\left(\text{th}_\text{i},\text{th}_\text{i}'\right)/\tau}}{\sum_{\text{j}=1}^{n} \text{e}^{\text{sim}(\text{th}_\text{i},\text{th}_\text{j})/\tau}} \tag{5}$$

Where τ is a temperature hyperparameter, and sim(h_1, h_2) represents the cosine similarity between vectors h_1 and h_2 [26]. This loss function optimizes contrastive learning by increasing the similarity between positive pairs and reducing it between negative pairs, thereby improving the consistency of the representation space and enhancing feature discrimination for service function vectors. The final service function vector sfv_i of m_i is set as the value of th_i when $\mathcal{L}_{fv}$ converges.

By applying contrastive learning, we align the functional semantics of service descriptions in positive sample pairs, improving the quality of service function feature extraction. Moreover,

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

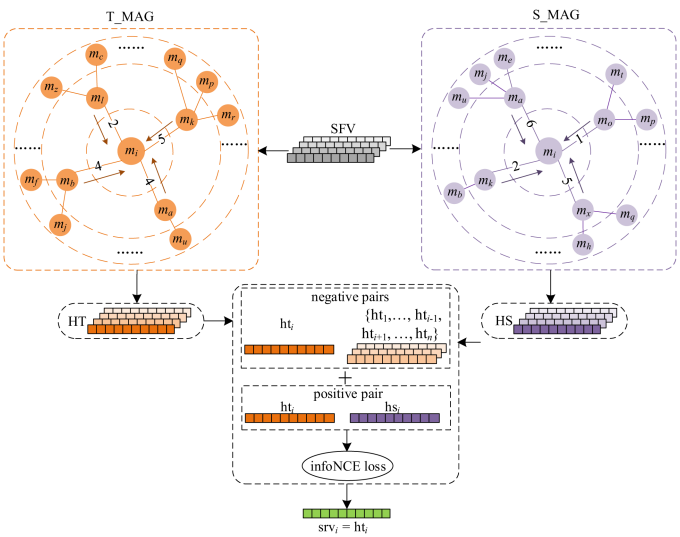


Fig. 5. Neighborhood feature aggregation leveraging contrastive learning across dual association graphs.

the differences between service descriptions in negative sample pairs enhance functional differentiation between services, optimizing the generation of service function vectors.

D. Service Neighborhood Feature Aggregation Via Contrastive Learning Across Dual Association Graphs

Leveraging graph neural network models to aggregate the functional features of neighboring nodes within a service association graph has emerged as a popular method to enhance the clustering quality of Mashup services. However, existing approaches have a notable limitation: while they effectively improve the quality of generated service function vectors by aggregating neighborhood features of service nodes, they fail to compare features among different neighboring nodes in this aggregation process. These methods merely extract functional features from associated nodes without selectively aggregating them based on the distinct characteristics of each neighboring node.

To address this shortcoming, we propose a service neighborhood feature aggregation method that uses contrastive learning across dual association graphs, as illustrated in Fig. 5. This method is grounded in the association dependencies that exist among Mashup services, giving rise to the construction of two distinct association graphs - one for Web service sharing (T_MAG) and the other for tag sharing (S_MAG).

Graph neural network models are employed to aggregate the neighborhood features from both service association graphs. The service representation vectors, derived from the same service node in different association graphs, serve as positive pairs for contrastive learning. Additionally, we construct negative pairs by service representation vectors of different service nodes within the same association graph. Contrastive learning is then employed to optimize the quality of Mashup service representation vectors.

In T_MAG and S_MAG, the initial vectors for Mashup service nodes are the service function vectors generated in above section. Let f_g represent a graph learning model used to aggregate neighborhood features of service nodes within the service association graphs. Equations (6) and (7) are applied to derive the service representation vector by aggregating the Mashup service m_i in T_MAG and S_MAG, respectively.

$$\mathrm{ht}_i = f_g\left(\mathrm{sfv}_i, \mathrm{T}_MAG\right) \tag{6}$$

$$\mathrm{hs}_i = f_g\left(\mathrm{sfv}_i, \mathrm{S}_MAG\right) \tag{7}$$

For Mashup service m_i, the vector ht_i obtained by aggregating neighborhood features in T_MAG and the vector hs_i, derived from neighborhood features in in S_MAG, are combined as the positive pair (ht_i, hs_i). The set of negative pairs $\{(\mathrm{ht}_i, \mathrm{ht}_j)\}$ is created from vectors ht_i and ht_j, where ht_j is aggregated from other Mashup services in T_MAG. Here, $1 <= i, j <= N_m$, and $i \neq j$, where N_m is the number of Mashup services. We then apply the following infoNCE loss function to iteratively update the trainable parameters.

$$\mathcal{L}_{rv} = -\log \frac{\mathrm{e}^{\mathrm{sim}(\mathrm{ht_i}, \mathrm{hs}_i)/\tau}}{\sum_{\mathrm{j=1}}^{\mathrm{N_M}} \mathrm{e}^{\mathrm{sim}(\mathrm{ht}_i, \mathrm{ht}_j)/\tau}} \tag{8}$$

The optimization objective and symbols are defined similarly to those in (5). Thus, the overall goal of the MSSCL clustering framework is to optimize the loss function as defined in (9). Once the loss function converges, the service representation vector srv_{i} for Mashup service m_i is obtained.

$$\mathcal{L} = \mathcal{L}_{fv} + \mathcal{L}_{rv} \tag{9}$$

Using contrastive learning, we leverage the semantic similarity of node aggregation features in positive sample pairs to increase the aggregation weights of features in nodes with similar graph structures. Conversely, the semantic differences between nodes in negative sample pairs are used to reduce the aggregation weights of features in nodes with significant structural differences, thereby optimizing the quality of neighborhood feature aggregation at the weight level.

E. Mashup Service Clustering method: BG-CL

We propose a Mashup service clustering method within the MSCCL framework, called BG-CL. In BG-CL, BERT is utilized to generate service function vectors, while an enhanced GAT model aggregates neighborhood features from both T_MAG and S_MAG, producing service representation vectors. These vectors are gathered into a set, *SRV*, which is subsequently used for service clustering.

While BERT is commonly used in natural language processing, we do not provide an introduction to it in this study. GAT, on the other hand, excels at differentiating the varying contributions of neighboring nodes by dynamically calculating attention weights to optimize the features of the target node [27], [28]. Compared to other graph representation learning models, such as GCN, GraphSAGE (Graph Sample and Aggregate), and GIN (Graph Isomorphism Networks), GAT offers superior feature aggregation, particularly in Mashup service association

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

graphs with high edge density. Therefore, GAT is the preferred model for neighborhood feature aggregation in this context.

In BG-CL, the service function vectors serve as the initial vectors for the service nodes in both T_MAG and S_MAG. As the GAT model aggregates neighborhood features for node v_i, it computes attention coefficients with neighboring nodes under the k-th attention head, as shown in (10).

$$\mathrm{e}_{i,j}^{k} = \sigma\left(W_e^k\left[\mathrm{sfv}_i \| \mathrm{sfv}_j\right] + \mathrm{b}_e^k\right) \tag{10}$$

Here, sfv_i and sfv_j represent the service function vectors of service node v_i and its neighbor node v_j, respectively. $W^k{}_e$ is a training matrix, and $b^k{}_e$ is the training bias. σ denotes the LeakyReLU activation function. The symbol "‖" is used to concatenate vectors. The attention coefficients are then normalized as (11).

$$\mathrm{aw}_{i,j}^{k} = \mathrm{softmax}(e_{i,j}^{k}) = \frac{\exp(e_{i,j}^{k})}{\sum_{p=1}^{N_i}\exp(e_{i,p}^{k})} \tag{11}$$

In (11), N_i represents the set of neighboring nodes of v_i. For each attention head, the normalized attention coefficient for each neighbor node of v_i is computed. (12) shows the final aggregated vector hg_i of node v_i.

$$\mathrm{hg_i} = \sigma\left(\frac{1}{\mathrm{K}}\sum_{k=1}^{K}\sum_{j\in N_i}\mathrm{aw}_{i,j}^{k}W^k\mathrm{sfv}_j\right) \tag{12}$$

Using (12), we can obtain the vectors of node v_i, which aggregate the neighborhood features in T_MAG and S_MAG, denoted as $\mathrm{ht_i}$ and $\mathrm{hs_i}$, respectively. Additionally, once Formula (9) converges, the service representation vector srv_i for node v_i is generated.

Although traditional GAT performs well in neighborhood feature aggregation, researchers have introduced a series of improved models, such as GAT-W [29], DRR-GAT [30], MC-GAT [31], and PMP-GAT [32], which outperform the original GAT. New GAT variants are continuously being proposed. To clarify, the contribution of our approach lies in proposing a framework to enhance the clustering quality of Mashup services. The framework's primary role is to improve the clustering performance of the integrated models, rather than focusing on identifying the most cutting-edge models.

To the best of our knowledge, PMP-GAT is a recently proposed variant of GAT that delivers excellent performance. In PMP-GAT, polarized message-passing (PMP) is introduced to enhance GAT by capturing both node-node similarity and dissimilarity, allowing it to gather dual sources of messages from neighboring nodes. These messages are then combined, enabling GAT to learn more expressive representations, particularly from sparse but strongly correlated neighbors. For this reason, we selected PMP-GAT as the neighborhood aggregation model in BG-CL.

F. Time Complexity

The time consumption of MSCCL primarily involves generating service function vectors, aggregating neighborhood features, and optimizing these processes through contrastive learning. Additionally, the running time of the clustering algorithm is also included.

Let the complexity of the model used to generate service function vectors be O(F), the complexity of the graph neural network model for neighborhood feature aggregation be O(G), and the time complexity of the clustering algorithm be O(C). After integrating these models into MSCCL, additional computational cost arises from contrastive learning. Suppose the number of Mashup services to be clustered is denoted by $|M|$. In contrastive learning, the number of negative samples for the service function vector is represented as $|Ns|$, while the number of negative samples during neighborhood feature aggregation is given by $|Ng|$. Consequently, the overall complexity of service clustering using MSCCL is expressed as O($F + |M| * |Ns| + 2 * G + |M| * |Ng| + C$).

TABLE I
OVERVIEW OF DATASET

Data	Service category	Number of services
DS1	Top-20	2270
DS2	Top-50	4101
DS3	Top-80	4773

IV. EXPERIMENTS

In this section, we conducted extensive experiments to answer the following research questions:

RQ1: Can the MSSCL effectively enhance the quality of Mashup service clustering?
RQ2: Does the MSSCL demonstrate broad applicability to widely used models for service function vector generation and neighborhood feature aggregation?
RQ3: Does the data augmentation method introduced in this paper outperform existing data augmentation techniques within the MSSCL?
RQ4: Is the Mashup service clustering method BG-CL superior to the state-of-the-art clustering approaches?
RQ5: Can large language models (LLMs) be integrated into MSSCL to produce high-quality service clustering?
RQ6: How do hyper-parameters affect the performance of service clustering (In appendix)?

A. Experimental Settings

We collected Mashup services from the Programmable Web. After filtering out services with excessively short descriptions and duplicate registrations, we retained 4773 Mashup services. To categorize these services, we engaged ten researchers, resulting in the creation of 80 distinct categories. Each researcher was asked to independently annotate the Mashup services. The final categories were determined by voting, with the category receiving the most votes assigned to each Mashup service. These Mashup services included 958 component Web services. The categories were ranked based on the number of Mashup services they contained.

As shown in Table I, we partitioned these Mashup services into three datasets by the incremental dataset construction

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE ABLATION STUDY ON MSSCL

Data	Model	DBI	SC	NMI	Purity
DS1	BG-CL	0.743	0.649	0.715	0.671
	BG-CLw/oFcl	**0.829**	**0.636**	**0.669**	**0.628**
	BG-CLw/oNcl	0.802	0.628	0.653	0.639
	BG-CLw/oFcl,Ncl	0.911	0.578	0.598	0.601
DS2	BG-CL	0.755	0.622	0.694	0.657
	BG-CLw/oFcl	**0.851**	**0.627**	**0.633**	**0.602**
	BG-CLw/oNcl	0.813	0.614	0.648	0.611
	BG-CLw/oFcl,Ncl	0.896	0.571	0.588	0.589
DS3	BG-CL	0.725	0.647	0.693	0.662
	BG-CLw/oFcl	**0.819**	**0.599**	**0.639**	**0.625**
	BG-CLw/oNcl	0.804	0.589	0.654	0.619
	BG-CLw/oFcl,Ncl	0.903	0.567	0.543	0.579

method commonly used in service clustering. Details regarding key hyperparameter settings are provided in the appendix. K-means++ algorithm is employed to cluster Mashup services.

All service description texts underwent preprocessing steps, including case conversion, word segmentation, stop word removal, and stemming, prior to Mashup service clustering. The quality of service clustering is evaluated using the following metrics: Davies-Bouldin Index (DBI), Silhouette Coefficient (SC), Normalized Mutual Information (NMI), and Purity. Notably, DBI is a negative evaluation metric, where lower values indicate better clustering quality. In contrast, higher values for SC, NMI, and Purity correspond to better clustering performance. The formal definitions of these metrics can be found in the appendix.

B. Experimental Results and Analysis

1) Effectiveness of MSSCL (RQ1): We conduct ablation experiments by selectively removing contrastive learning from the service function vector generation module and the neighborhood feature aggregation module in MSSCL. To assess the effectiveness of MSSCL, we analyze the variations in service clustering quality using BG-CL. Specifically, the variants are designated as follows:

a) *BG-CLw/oFcl:* Contrastive learning in the service function vector generation module is disabled.
b) *BG-CLw/oNcl:* Contrastive learning in the neighborhood feature aggregation module is removed.
c) *BG-CLw/oFcl,Ncl:* Contrastive learning is eliminated from both the service function vector generation module and the neighborhood feature aggregation module.

Table II shows the results of the ablation study on MSSCL. The data in Table II indicate that removing contrastive learning from either the service function vector generation module or the neighborhood feature aggregation module results in a decline in clustering quality. This underscores the importance of contrastive learning in enhancing clustering performance within the MSSCL framework. Notably, when contrastive learning is removed from both modules, the degradation in Mashup service clustering quality is significantly more pronounced than when it is removed from only one module. These findings suggest that MSSCL effectively improves clustering quality by leveraging contrastive learning in both the service function vector generation and neighborhood feature aggregation processes, demonstrating its robustness in service clustering optimization.

TABLE III
QUALITY IMPROVEMENT OF SERVICE FUNCTION VECTORS BY CONTRASTIVE LEARNING

Data	Model	DBI	SC	NMI	Purity
DS1	LDA	1.683	0.353	0.343	0.311
	LDA-CL	1.527	0.369	0.359	0.322
	GSDMM	0.968	0.472	0.508	0.496
	GSDMM-CL	0.947	0.489	0.567	0.56
	Doc2Vec	0.997	0.507	0.516	0.537
	Doc2Vec-CL	0.938	0.575	0.567	0.561
	BERT	0.937	0.516	0.559	0.554
	BERT-CL	**0.892**	**0.582**	**0.628**	**0.617**
DS2	LDA	1.809	0.330	0.316	0.299
	LDA-CL	1.721	0.347	0.342	0.329
	GSDMM	0.964	0.468	0.478	0.473
	GSDMM-CL	0.941	0.482	0.533	0.523
	Doc2Vec	0.958	0.521	0.502	0.504
	Doc2Vec-CL	0.904	0.543	0.548	0.542
	BERT	0.922	0.519	0.553	0.532
	BERT-CL	**0.876**	**0.554**	**0.605**	**0.573**
DS3	LDA	1.698	0.332	0.309	0.302
	LDA-CL	1.534	0.356	0.344	0.326
	GSDMM	0.961	0.480	0.484	0.488
	GSDMM-CL	0.943	0.497	0.559	0.532
	Doc2Vec	0.922	0.496	0.507	0.515
	Doc2Vec-CL	0.894	0.543	0.551	0.545
	BERT	0.923	0.514	0.542	0.539
	BERT-CL	**0.867**	**0.562**	**0.603**	**0.571**

2) Generalization of MSSCL(RQ2): In this section, we independently evaluate the effectiveness of replacing the models used in the service function vector generation module and the neighborhood feature aggregation module of MSSCL.

In the service function vector module, we select four commonly used models from existing research: LDA [33], GSDMM [34], Doc2Vec [35], and BERT [36]. To assess whether contrastive learning improves the quality of service function vectors, we compared the clustering performance of each model before and after integrating contrastive learning. The results, shown in Table III, display the scores of the service clustering evaluation metrics. Models with the "CL" suffix represent those incorporating contrastive learning.

Table III reveals that, across different datasets, contrastive learning consistently leads to improvements in all clustering evaluation metrics. For LDA, on average, DBI decreased by 7.86%, while SC, NMI, and Purity increased by an average of 5.61%, 7.95%, and 7.13%, respectively. For GSDMM, DBI saw an average reduction of 2.14%, and SC, NMI, and Purity increased by an average of 3.38%, 12.85%, and 10.84%, respectively. Doc2Vec exhibited an average DBI reduction of 4.9%, with SC, NMI, and Purity increasing by an average of 8.98%, 9.25%, and 5.91%, respectively. Lastly, BERT showed an average DBI reduction of 5.28%, and SC, NMI, and Purity increased by an average of 9.62%, 11%, and 8.37%, respectively.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

The experimental results demonstrate that when the aforementioned models are integrated into MSSCL, the service function vectors they generate consistently lead to improved clustering quality. This confirms that the contrastive learning-based service function vector module in MSSCL is effective across commonly used models and exhibits strong generalizability. Furthermore, we observe that among all tested models, BERT achieves the highest service clustering performance, which is why we select BERT for generating service function vectors in BG-CL.

We selected six comparison models, including GCN [37], Graphsage [38], GIN [39], GAT [27], GAT-W [29] and PMP-GAT [32], to investigate whether the introduction of contrastive learning enhances the quality of neighborhood feature aggregation. The above models typically aggregate neighborhood features using a single service association graph. However, this study constructs two association graphs, T_MAG and S_MAG, and employs contrastive learning to optimize neighborhood feature aggregation across both graphs.

BERT-CL was used to generate service function vectors, which were then utilized for clustering without neighborhood feature aggregation. The evaluation metrics of these clusters serve as a baseline, shown in the first row of each dataset in Table IV (rows labeled "BERT-CL" under the "Model" column). For each model, we conducted three sets of experiments: a) neighborhood feature aggregation in T_MAG, b) neighborhood feature aggregation in S_MAG, and c) neighborhood feature aggregation with contrastive learning across dual association graphs (DAG). In this study, the models were trained in an unsupervised manner using the negative sampling loss function of GraphSAGE in the case of single association graphs.

Table IV shows that service clustering quality improves significantly across all models when neighborhood feature aggregation is applied, whether using S_MAG, T_MAG, or DAG. Moreover, when contrastive learning is employed within DAG, the clustering performance is further enhanced compared to using a single association graph. This indicates that the contrastive learning approach we designed for DAG is highly effective in improving the quality of neighborhood node aggregation. This demonstrates that the neighborhood feature aggregation module in MSSCL exhibits strong generalizability.

Notably, GATs outperformed GCN, Graphsage, and GIN in terms of the quality of neighborhood feature aggregation. The primary factor contributing to this superiority is their attention mechanism, which enables them to distinguish the varying contributions of neighboring nodes, leading to more effective aggregation of neighborhood features [40], [41].

Compared to the original GAT, the improved GATs deliver higher-quality service clustering. GAT-W enhances clustering by incorporating edge weights, allowing for more accurate differentiation in feature contributions from different neighboring nodes. PMP-GAT improves GAT by introducing polarized message-passing, which captures both node-node similarity and dissimilarity, enabling the model to gather dual sources of information from neighboring nodes. Across all metrics, PMP-GAT achieved the highest performance, making it the preferred model for implementing our proposed Mashup service clustering method BG-CL.

TABLE IV
EFFECT OF CONTRASTIVE LEARNING ON AGGREGATION OF NEIGHBORHOOD FEATURE

Data	Model	Association mode	DBI	SC	NMI	Purity
DS1	BERT-CL		0.892	0.582	0.628	0.617
	BERT-CL-GCN	S_MAG	0.838	0.593	0.635	0.621
		T_MAG	0.835	0.597	0.633	0.623
		DAG	**0.772**	**0.609**	**0.653**	**0.643**
	BERT-CL-Graphsage	S_MAG	0.848	0.611	0.658	0.635
		T_MAG	0.843	0.613	0.665	0.632
		DAG	**0.771**	**0.618**	**0.688**	**0.650**
	BERT-CL-GIN	S_MAG	0.848	0.602	0.642	0.609
		T_MAG	0.837	0.597	0.646	0.616
		DAG	**0.787**	**0.611**	**0.659**	**0.626**
	BERT-CL-GAT	S_MAG	0.835	0.571	0.632	0.596
		T_MAG	0.837	0.575	0.629	0.598
		DAG	**0.751**	**0.621**	**0.662**	**0.640**
	BERT-CL-GAT-W	S_MAG	0.803	0.605	0.649	0.630
		T_MAG	0.814	0.616	0.645	0.634
		DAG	**0.745**	**0.638**	**0.703**	**0.662**
	BERT-CL-GAT-P	S_MAG	0.795	0.616	0.652	0.637
		T_MAG	0.802	0.628	0.653	0.639
		DAG	**0.743**	**0.649**	**0.715**	**0.671**
DS2	BERT-CL		0.876	0.554	0.605	0.573
	BERT-CL-GCN	S_MAG	0.842	0.570	0.621	0.583
		T_MAG	0.844	0.572	0.627	0.584
		DAG	**0.783**	**0.588**	**0.647**	**0.612**
	BERT-CL-Graphsage	S_MAG	0.847	0.585	0.633	0.593
		T_MAG	0.849	0.589	0.632	0.602
		DAG	**0.778**	**0.607**	**0.654**	**0.624**
	BERT-CL-GIN	S_MAG	0.848	0.586	0.624	0.6
		T_MAG	0.853	0.591	0.623	0.597
		DAG	**0.802**	**0.604**	**0.636**	**0.618**
	BERT-CL-GAT	S_MAG	0.834	0.576	0.622	0.585
		T_MAG	0.835	0.579	0.618	0.586
		DAG	**0.772**	**0.587**	**0.659**	**0.624**
	BERT-CL-GAT-W	S_MAG	0.832	0.591	0.642	0.594
		T_MAG	0.829	0.596	0.637	0.601
		DAG	**0.762**	**0.617**	**0.682**	**0.648**
	BERT-CL-GAT-P	S_MAG	0.821	0.608	0.650	0.604
		T_MAG	0.813	0.614	0.648	0.611
		DAG	**0.755**	**0.622**	**0.694**	**0.657**
DS3	BERT-CL		0.867	0.562	0.603	0.571
	BERT-CL-GCN	S_MAG	0.823	0.569	0.628	0.595
		T_MAG	0.816	0.574	0.625	0.597
		DAG	**0.752**	**0.617**	**0.652**	**0.633**
	BERT-CL-Graphsage	S_MAG	0.842	0.583	0.637	0.599
		T_MAG	0.837	0.587	0.643	0.605
		DAG	**0.761**	**0.602**	**0.664**	**0.625**
	BERT-CL-GIN	S_MAG	0.814	0.573	0.634	0.588
		T_MAG	0.795	0.577	0.641	0.592
		DAG	**0.769**	**0.595**	**0.658**	**0.611**
	BERT-CL-GAT	S_MAG	0.823	0.591	0.626	0.626
		T_MAG	0.812	0.594	0.633	0.627
		DAG	**0.763**	**0.61**	**0.669**	**0.644**
	BERT-CL-GAT-W	S_MAG	0.808	0.576	0.643	0.605
		T_MAG	0.815	0.578	0.639	0.608
		DAG	**0.728**	**0.634**	**0.685**	**0.654**
	BERT-CL-GAT-P	S_MAG	0.796	0.585	0.652	0.613
		T_MAG	0.804	0.589	0.654	0.619
		DAG	**0.725**	**0.647**	**0.693**	**0.662**

3) Advancement of Data Augmentation Methods (RQ3): We have designed data augmentation methods for the two contrastive learning components in MSSCL. Text data augmentation

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

TABLE V
COMPARISON OF CLUSTERING PERFORMANCE BETWEEN VSS AND OTHER TEXT DATA AUGMENTATION METHODS

Data	Method	DBI	SC	NMI	Purity
DS1	RD	0.933	0.548	0.609	0.554
	BT	0.915	0.562	0.609	0.578
	RSS	0.906	0.567	0.612	0.582
	DTR	0.898	0.575	0.614	0.589
	VSS	**0.892**	**0.582**	**0.628**	**0.617**
DS2	RD	0.956	0.501	0.538	0.553
	BT	0.913	0.549	0.571	0.563
	RSS	0.910	0.544	0.582	0.566
	DTR	0.885	0.551	0.595	0.570
	VSS	**0.876**	**0.554**	**0.605**	**0.573**
DS3	RD	0.942	0.524	0.592	0.549
	BT	0.896	0.546	0.594	0.566
	RSS	0.899	0.549	0.598	0.569
	DTR	0.872	0.552	0.601	0.568
	VSS	**0.867**	**0.562**	**0.603**	**0.571**

is used in the service function vector generation module. Common text data augmentation methods include Random Deletion (RD), Back Translation (BT), Random Synonym Substitution (RSS) [25], [42], and a novel approach recently introduced [43], which involves applying Dropout twice to BERT neurons (DTR).

We first assess whether our proposed Verb Synonym Substitution (VSS) method surpasses RD, BT, RSS, and DTR in performance. BERT was employed in this experiment. The comparison of clustering performance across these text data augmentation techniques is presented in Table V.

Table V reveals that among all the data augmentation methods, RD performs the poorest. Given that Mashup service descriptions are typically short texts with sparse features, RD's tendency to remove feature words can significantly impact the service function vectors. Both BT and RSS share similar principles. However, the translation quality of BT is somewhat unstable and can introduce noise, making its performance slightly inferior to RSS [25]. The DTR method, which leverages the random inactivation characteristics of BERT neurons during training to construct positive sample pairs, excels in preserving semantic information, achieving superior scores across various evaluation metrics. These methods exhibit a high degree of randomness and frequently alter the service functional characteristics, especially those identified by nouns, which is not conducive to clustering Mashup services with similar functions.

The VSS method proposed in this paper achieved the highest service clustering quality.VSS selectively replaces verbs with weaker functional relevance, thereby better preserving the core service characteristics and enhancing the quality of contrastive learning and clustering. Thus, when generating service function vectors, the VSS method proposed in this study outperforms other commonly used text data augmentation techniques.

In the neighborhood feature aggregation module, we employ graph data augmentation. Common graph data augmentation methods for contrastive learning include Random Node Deletion (RDN) [44], Random Edge Deletion (RDE) [45], and Random Walk Sampling (RWS) [46].

TABLE VI
COMPARISON OF CLUSTERING QUALITY BETWEEN THE PROPOSE GRAPH DATA AUGMENTATION METHOD AND OTHER GRAPH DATA AUGMENTATION METHODS

Data	Method	DBI	SC	NMI	Purity
DS1	RDN	0.786	0.588	0.663	0.651
	RDE	0.757	0.595	0.657	0.640
	RWS	0.747	0.634	0.692	0.658
	DAG	**0.743**	**0.649**	**0.715**	**0.671**
DS2	RDN	0.801	0.586	0.607	0.597
	RDE	0.785	0.591	0.616	0.603
	RWS	0.768	0.612	0.679	0.641
	DAG	**0.755**	**0.622**	**0.694**	**0.657**
DS3	RDN	0.790	0.622	0.669	0.633
	RDE	0.762	0.617	0.671	0.636
	RWS	0.729	0.631	0.677	0.650
	DAG	**0.725**	**0.647**	**0.693**	**0.662**

We compare the clustering quality of service representation vectors obtained through the GAT-P using different graph data augmentation techniques in two scenarios: a) a single association graph (T_MAG) and b) the proposed dual association graphs (DAG). Our goal is to demonstrate the superiority of the DAG-based augmentation method.

Among the methods listed in Table VI, RDN is the least effective. This is due to the high edge density in T_MAG, where removing a node results in the deletion of many edges, significantly altering neighborhood structure and degrading the quality of feature aggregation. On the other hand, RDE and RWS, which do not involve node removal, preserve more of the original graph structure, leading to better performance than RDN. Notably, RWS excels by identifying additional link structures and retaining more information from the original graph. Compared to other graph data augmentation methods, our approach achieved a greater reduction in DBI and improvements in SC, NMI, and Purity.

The proposed DAG method constructs two types of service association graphs, maintaining the original graph structure while aggregating neighborhood features from two different association dimensions. This significantly improves the quality of neighborhood feature aggregation, leading to superior performance over the other methods. As presented in Table VI, the proposed DAG-based augmentation method outperforms state-of-the-art techniques on all datasets.

4) Clustering Performance of BG-CL (RQ4): We compare the performance of BG-CL with the following methods:

a) *AUG-LDA [19]:* This method clusters terms within service descriptions and constructs an AUG-LDA model, utilizing term clusters as auxiliary information to get hidden topic vectors from service descriptions. Service clustering is then accomplished using the K-means++ algorithm.
b) *PICF-LDA [13]:* This method introduces a probability correction factor to enhance the topic extraction capabilities of LDA. Service function vectors are generated by PICF-LDA and input into the K-means algorithm for clustering.
c) *Sim-LDA-SC [23]:* This method integrates structural similarity and semantic similarity through a fuzzy analytic hierarchy process. Structural similarity is derived from usage histories between Mashups and Web APIs, while

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

semantic similarity is generated from Mashup descriptions. Service clustering is performed using a genetic algorithm.

d) *S3C [11]:* This method expands the feature set of service descriptions by using the RGPS (Role-Goal-Process-Service) meta-model framework. Latent service features are extracted using the Bi-term Topic Model (BTM), and service clustering is performed using K-means algorithm.

e) *TWE-NMF [20]:* This approach leverages GSDMM to estimate the number of topics automatically. It combines word embeddings and service tag information, using non-negative matrix factorization to compute Mashup topic features. Spectral clustering algorithm is then applied for Mashup service clustering.

f) *WGSDMM+GA [12]:* This method employs Word2Vec and GSDMM to obtain the features and semantic relationships for Web services. A genetic clustering method based on the Manhattan distance is then applied to cluster Web services.

g) *TEDM [47]:* This method uses dynamic clustering, leveraging topic information to improve the sampling of documents and better cluster documents on the same topic. It constructs a dynamic word relation graph to extract topic terms, which is updated with the stream of documents to cope with the dynamic changes in topics.

h) *WSNE [48]:* This method introduces an attention-based relational topic model to extract latent topics from service descriptions. It incorporates a service graph convolutional network to merge network structure and content information, enabling Web service representation learning across the Mashup-API network. Service clustering is conducted using the K-means++ algorithm.

i) *SGL [49]:* This method proposes a scalable graph learning framework. The authors construct a bipartite graph to depict the relationship between samples and anchor points. Meanwhile, a connectivity constraint is employed to ensure that the connected components indicate clusters directly.

TABLE VII
COMPARISON OF BG-CL WITH OTHER CLUSTERING METHODS

Data	Model	DBI	SC	NMI	Purity
DS1	AUG-LDA	0.838	0.564	0.644	0.586
	PICF-LDA	0.819	0.595	0.658	0.598
	Sim-LDA-SC	0.821	0.601	0.662	0.624
	S3C	0.802	0.603	0.655	0.632
	TWE-NMF	0.781	0.62	0.674	0.637
	WGSDMM+ GA	0.769	0.621	0.677	0.639
	TEDM	0.756	0.619	0.673	0.640
	WSNE	**0.742**	0.618	0.678	0.636
	SGL	0.744	0.627	0.683	0.645
	BG-CL	0.743	**0.649**	**0.715**	**0.671**
DS2	AUG-LDA	0.861	0.538	0.602	0.580
	PICF-LDA	0.84	0.547	0.599	0.585
	Sim-LDA-SC	0.833	0.565	0.604	0.597
	S3C	0.818	0.573	0.617	0.606
	TWE-NMF	0.796	0.593	0.615	0.622
	WGSDMM+ GA	0.783	0.599	0.639	0.634
	TEDM	0.781	0.603	0.642	0.636
	WSNE	0.775	0.605	0.646	0.638
	SGL	0.762	0.610	0.667	0.646
	BG-CL	**0.755**	**0.622**	**0.694**	**0.657**
DS3	AUG-LDA	0.844	0.559	0.641	0.585
	PICF-LDA	0.825	0.563	0.647	0.587
	Sim-LDA-SC	0.816	0.588	0.643	0.609
	S3C	0.809	0.609	0.649	0.624
	TWE-NMF	0.784	0.608	0.654	0.635
	WGSDMM+ GA	0.782	0.612	0.67	0.639
	TEDM	0.784	0.616	0.672	0.642
	WSNE	0.786	0.614	0.677	0.641
	SGL	0.763	0.639	0.684	0.649
	BG-CL	**0.725**	**0.647**	**0.693**	**0.662**

Table VII compares the performance of BG-CL with several state-of-the-art methods, highlighting the best results in bold. BG-CL achieves the top scores across all datasets, indicating its superior quality in Mashup service clustering. Specifically, BG-CL reduces the DBI by 2.03% to 12.58% compared to other methods, while improving the SC by 2.24% to 15.47%. These results suggest that BG-CL produces service clusters with more closely related services and greater distances between clusters. Furthermore, the NMI and Purity improve by 3.34% to 11.39% and 2.58% to 13.65%, respectively, demonstrating better alignment between the clustering results and actual service categories.

Among these methods, AUG-LDA yields the lowest quality scores. It uses term clustering in the service description to enhance the latent semantic extraction. The improvement of clustering accuracy is limited, due to the sparse density of functional feature words in the service description. PICF-LDA proposes the probability incremental correction factor to rectify the probability distributions of functional features in the service function vector. It performs better than AUG-LDA. Sim-LDA-SC, which integrates service sharing associations and calculates functional semantic similarity among Mashup services, offers superior clustering quality compared to AUG-LDA and PICF-LDA.

S3C leverages RGPS to expand service description features and employs BTM for service function vector generation. Its combination of an advanced topic model and an extended feature set leads to better clustering performance than previous methods. WGSDMM+GA performs well in service clustering by using GSDMM, which is more suitable for short-text feature extraction than LDA and BTM. TWE-NMF, while also employing GSDMM, enhances its performance by integrating service tags and text context information, but it falls short of WSNE and BG-CL due to its lack of consideration for Mashup-Web API interactions.

WSNE employs the attention mechanism to enhance the relational topic model, leading to an improved quality in service function vector generation. It learns potential interaction characteristics based on the invocation relationships of Mashup-Web services, resulting in a stronger clustering capacity than the aforementioned methods.

Some clustering algorithms [47], [49], [50], [51], [52], [53] from non-service computing fields offer valuable insights for our research. For instance, TEDM and SGL are both innovative and advanced clustering methods, demonstrating excellent performance in their respective domains. TEDM excels in dynamic clustering of text streams, while SGL is advantageous for

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

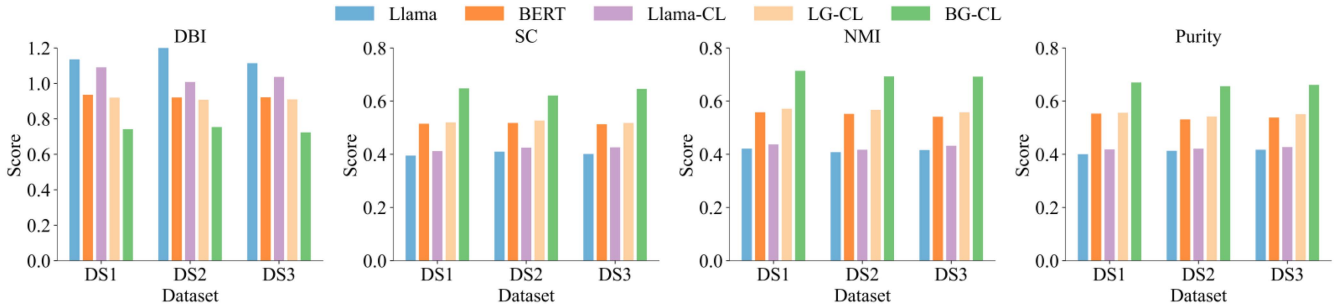


Fig. 6. Comparisons of service clustering quality using Llama-7B and BERT.

graph-based subspace clustering. Additionally, Chao et al. [54] proposes a novel multi-view deep subspace clustering method using contrastive learning and Cauchy-Schwarz divergence, combining representation learning with clustering to enable their interaction. However, when applied to Mashup service clustering, they primarily focus on extracting features from service description text, without directly accounting for the collaboration between component services. As a result, their final clustering quality is slightly lower than our proposed method.

In contrast, BG-CL surpasses all others by using BERT to generate service function vectors, outperforming traditional topic models. Additionally, BG-CL incorporates Mashup service associations to further enhance clustering performance. Most importantly, BG-CL introduces contrastive learning in both service function vector generation and neighborhood structure feature aggregation, significantly improving feature discrimination and aggregation. As a result, BG-CL achieves the best clustering performance among all tested methods.

5) LLMs and MSSCL (RQ5): In this section, we will investigate the integration of large language models with MSSCL, focusing primarily on the following issues:

a) Can LLMs generate high-quality service function vectors for service clustering?
b) Can LLMs be integrated into MSSCL, and how effective is the service clustering performance after integration?
c) How can LLMs be leveraged to enhance service clustering quality?

Prominent LLMs, including GPT, Llama, and DeepSeek, feature vast parameters. They have accumulated extensive knowledge through pretraining. We do not use these large models to generate service function vectors in the clustering algorithm BG-CL. Instead, we employed BERT, which, while still a large model, has significantly fewer parameters than the aforementioned LLMs.

To investigate the first two questions, we conducted experiments. Due to API accessibility constraints and computational limitations, we selected Llama-7B as the comparative LLM. Fig. 6 presents comparisons of service clustering quality using Llama-7B, BERT, and their respective integrations into MSSCL.

Experimental results reveal that the quality service function vectors generated Llama-7B is lower than those obtained by BERT. This disparity arises because BERT, a bidirectional encoder model based on the Transformer architecture, excels at producing high-quality semantic vector representations for text. Conversely, Llama, a unidirectional autoregressive model built on the Transformer architecture, differs significantly from BERT. During text generation, Llama follows a left-to-right sequence, progressively generating tokens based on previously generated ones while sampling from a probability distribution over the vocabulary. While this approach enhances Llama's ability to handle long-form text generation, it is less effective at generating high-quality semantic vectors for short service descriptions. Moreover, existing studies have demonstrated that LLMs excel in generative tasks, whereas their ability to extract semantic features from text tends to be relatively weaker [55], [56]. This finding aligns with the results of our experiments.

We define the method of generating service function vectors by combining Llama-7B with contrastive learning as Llama-CL, and the method of embedding Llama into MSSCL as LG-CL. As shown in Fig. 6, both Llama-CL and LG-CL demonstrate improved clustering performance, confirming that MSSCL is effective for Llama. However, since the quality service function vectors generated Llama-7B is lower than those obtained by BERT, LG-CL performs worse compared to our proposed BG-CL method.

Regarding the third question, our experiments indicate that directly employing LLMs for service function vector generation may not enhance service clustering quality. This is primarily because LLMs are optimized for task generation rather than extracting high-quality semantic representations. Notably, Mashup service descriptions tend to be short, further limiting the effectiveness of LLM-generated function vectors. These observations justify our decision to use BERT in our experiments, which led to superior clustering results.

Although directly using LLMs to generate service function vectors does not yield high-quality service clustering, we believe that the knowledge accumulated by these models during pre-training can assist other semantic extraction models in more effectively capturing service functions, thereby improving clustering quality. However, the knowledge in LLMs is implicit, and integrating this knowledge with service descriptions, as well as determining the optimal amount to incorporate, presents significant challenges. These are key issues we aim to address in our future research.

V. Conclusion

In this paper, we introduce MSSCL, a novel framework designed to enhance the clustering quality of Mashup services. By incorporating contrastive learning, MSSCL improves both

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

the generation of service function vectors and the aggregation of neighborhood features. Extensive experiments on public datasets demonstrate that various service clustering models can be effectively integrated into MSSCL. Moreover, the results indicate a significant improvement in clustering quality after embedding MSSCL compared to before, highlighting its effectiveness and generalization.

Future work will explore the integration of LLMs or knowledge graphs to further enhance the quality of service function vector generation. Additionally, we aim to incorporate more service associations into neighborhood feature aggregation module to achieve even more accurate and robust service clustering.

References

[1] K. Zeng and I. Paik, "Web service embedding: Representing the invocation association between services with practical-valued vectors," *Expert Syst. Appl.*, vol. 238, no. 15, Mar. 2024, Art. no. 122196, doi: 10.1016/j.eswa.2023.122196.

[2] N. Agarwal, G. Sikka, and L. K. Awasthi, "Integrating semantic similarity with Dirichlet multinomial mixture model for enhanced web service clustering," *Knowl. Inf. Syst.*, vol. 66, no. 4, pp. 2327–2353, Dec. 2024, doi: 10.1007/s10115-023-02034-x.

[3] B. Cao et al., "Web service recommendation via integrating heterogeneous graph attention network representation and FiBiNET score prediction," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3837–3850, Sep./Oct. 2023, doi: 10.1109/TSC.2023.3287189.

[4] Q. Hu, J. Shen, K. Wang, J. Du, and Y. Du, "A web service clustering method based on topic enhanced Gibbs sampling algorithm for the Dirichlet Multinomial Mixture model and service collaboration graph," *Inform. Sci.*, vol. 586, pp. 239–260, Mar. 2022, doi: 10.1016/j.ins.2021.11.087.

[5] N. Agarwal, G. Sikka, and L. K. Awasthi, "Web service clustering technique based on contextual word embedding for service representation," in *Proc. Int. Conf. Technological Advancements Innovations*, Portland, OR, USA, 2021, pp. 617–621.

[6] M. Shi, Y. Tang, and J. Liu, "Functional and contextual attention-based LSTM for service recommendation in mashup creation," *IEEE Trans. Parallel Distrb. Syst.*, vol. 30, no. 5, pp. 1077–1090, May 2019, doi: 10.1109/TPDS.2018.2877363.

[7] S. Jalal, D. K. Yadav, and C. S. Negi, "Web service discovery with incorporation of web services clustering," *Int. J. Comput. Appl.*, vol. 45, no. 1, pp. 51–62, Aug. 2023, doi: 10.1080/1206212X.2019.1698131.

[8] L. Purohit, S. Rathore, and S. Kumar, "A QoS-aware clustering based multi-layer model for web service selection," *IEEE Trans. Serv. Comput.*, vol. 16, no. 5, pp. 3141–3154, Sep./Oct. 2023, doi: 10.1109/TSC.2023.3264627.

[9] B. Cao, L. Zhang, M. Peng, Y. Qing, G. Kang, and J. Liu, "Web service recommendation via combining bilinear graph representation and xDeepFM quality prediction," *IEEE Trans. Netw. Serv.*, vol. 20, no. 2, pp. 1078–1092, Jun. 2023, doi: 10.1109/TNSM.2023.3234067.

[10] D. Zhang, F. Nan, X. Wei, S. Li, H. Zhu, and K. McKeown, "Supporting clustering with contrastive learning," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, Mexico, CDMX, Mexico, 2021, pp. 5419–5430.

[11] F. Xie, J. Chen, Y. Zhu, and H. Zheng, "A web service clustering method with semantic enhancement based on RGPS and BTM," *J. Internet Technol.*, vol. 24, no. 4, pp. 945–953, 2023, doi: 10.53106/160792642023072404012.

[12] N. Agarwal, G. Sikka, and L. K. Awasthi, "WGSDMM+GA: A genetic algorithm-based service clustering methodology assimilating dirichlet multinomial mixture model with word embedding," *Future Gener. Comput. Syst.*, vol. 145, pp. 254–266, 2023, doi: 10.1016/j.future.2023.03.028.

[13] J. Shen, W. Huang, and Q. Hu, "PICF-LDA: A topic enhanced LDA with probability incremental correction factor for web API service clustering," *J. Cloud Comput.: Adv. Syst. Appl.*, vol. 11, no. 1, pp. 1–13, Jul. 2022, doi: 10.1186/s13677-022-00291-9.

[14] Q. Hu, H. Qi, W. Huang, and M. Liu, "A method to recommend cloud manufacturing service based on the spectral clustering and improved Slope one algorithm," *J. Cloud Comput.: Adv. Syst. Appl.*, vol. 12, no. 1, pp. 1–17, Sep. 2023, doi: 10.1186/s13677-023-00489-5.

[15] G. Kang, J. Liu, Y. Xiao, Y. Cao, B. Cao, and M. Shi, "Web services clustering via exploring unified content and structural semantic representation," *IEEE Trans. Netw Serv.*, vol. 19, no. 4, pp. 4082–4096, Dec. 2022, doi: 10.1109/TNSM.2022.3197725.

[16] D. Ping, G. Bing, and Z. Wen, "Web service clustering approach based on network and fused document-based and tag-based topics similarity," *Int. J. Web Serv. Res.*, vol. 18, no. 3, pp. 63–81, 2021, doi: 10.4018/IJWSR.2021070104.

[17] G. Zou, Z. Qin, Q. He, P. Wang, B. Zhang, and Y. Gan, "DeepWSC: Clustering web services via integrating service composability into deep semantic features," *IEEE Trans. Serv. Comput.*, vol. 15, no. 4, pp. 1940–1953, Sep. 2020, doi: 10.1109/TSC.2020.3026188.

[18] M. M. Rahman, X. Liu, and B. Cao, "Web API recommendation for mashup development using matrix factorization on integrated content and network-based service clustering," in *Proc. IEEE Int. Conf. Serv. Comput.*, Honolulu, HI, USA, 2017, pp. 225–232.

[19] T. Ramathulasi and M. Rajasekharababu, "Augmented latent dirichlet allocation model via word embedded clusters for mashup service clustering," *Concurrency Computation: Pract. Experience*, vol. 34, no. 15, Feb. 2022, Art. no. e6896, doi: 10.1002/cpe.6896.

[20] J. Lu, W. Zhao, Y. Zhang, Q. Liang, and G. Xiao, "TWE-NMF topic model-based approach for mashup service clustering," *J. Softw.*, vol. 34, no. 6, pp. 2727–2748, 2022, doi: 10.13328/j.cnki.jos.006508.

[21] B. Cao, X. Liu, J. Liu, and M. Tang, "Domain-aware mashup service clustering based on LDA topic model from multiple data sources," *Inf. Softw. Technol.*, vol. 90, pp. 40–54, Oct. 2017, doi: 10.1016/j.infsof.2017.05.001.

[22] W. Pan and C. Chai, "Structure-aware mashup service clustering for cloud-based Internet of Things using genetic algorithm based clustering algorithm," *Future Gener. Comput. Syst.*, vol. 87, pp. 267–277, Oct. 2018, doi: 10.1016/j.future.2018.04.052.

[23] W. Pan, X. Xu, H. Ming, and C. Chang, "Clustering mashups by integrating structural and semantic similarities using fuzzy AHP," *Int. J. Web Serv. Res.*, vol. 18, no. 1, pp. 34–57, 2021, doi: 10.4018/IJWSR.2021010103.

[24] Y. Li, P. Hu, Z. Liu, D. Peng, J. Zhou, and X. Peng, "Contrastive clustering," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 8547–8555.

[25] J. Wei and K. Zou, "EDA: Easy data augmentation techniques for boosting performance on text classification tasks," in *Proc. Int. Joint Conf. Natural Lang. Process.*, Hong Kong, China, 2019, pp. 3–7.

[26] Y. Zhang, Y. Wada, H. Waida, K. Goto, Y. Hino, and T. Kanamori, "Deep clustering with a constraint for topological invariance based on symmetric infoNCE," *Neural Comput.*, vol. 35, pp. 1288–1339, Jun. 2023, doi: 10.1162/neco_a_01591.

[27] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2018, *arXiv:1710.10903*.

[28] L. He, L. Bai, X. Yang, H. Du, and J. Liang, "High-order graph attention network," *Inf. Sci.*, vol. 630, pp. 222–234, Jun. 2023, doi: 10.1016/j.ins.2023.02.054.

[29] Q. Hu, H. Qi, H. Li, and J. Du, "Web API recommendation by fusing latent related words and heterogeneous association compatibility," *J. Softw.*, May 2025. [Online]. Available: https://www.jos.org.cn/jos/article/abstract/7185

[30] X. Zhang, C. Zhang, J. Guo, C. Peng, Z. Niu, and X. Wu, "Graph attention network with dynamic representation of relations for knowledge graph completion," *Expert Syst. Appl.*, vol. 219, pp. 119616, Jun. 2023, doi: 10.1016/j.eswa.2023.119616.

[31] Z. La, Y. Qian, H. Leng, T. Gu, W. Gong, and J. Chen, "MC-GAT: Multi-channel graph attention networks for capturing diverse information in complex graphs," *Cogn. Comput.*, vol. 16, no. 2, pp. 595–607, Nov. 2023, doi: 10.1007/s12559-023-10222-8.

[32] T. He, Y. Liu, Y.-S. Ong, X. Wu, and X. Luo, "Polarized message-passing in graph neural networks," *Artif. Intell.*, vol. 331, Jun. 2024, Art. no. 104129, doi: 10.1016/j.artint.2024.104129.

[33] Y. Zhao, Y. Qiao, and K. He, "A novel tagging augmented LDA model for clustering," *Int. J. Web Serv. Res.*, vol. 16, no. 3, pp. 59–77, 2019, doi: 10.4018/IJWSR.2019070104.

[34] J. Yin and J. Wang, "A Dirichlet multinomial mixture model-based approach for short text clustering," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discov. data Mining*, New York, NY, USA, 2014, pp. 233–242.

[35] A. M. Dakhel, M. C. Desmarais, and F. Khomh, "Dev2vec: Representing domain expertise of developers in an embedding space," *Inf. Softw. Technol.*, vol. 159, Jul. 2023, Art. no. 107218, doi: 10.1016/j.infsof.2023.107218.

[36] J. Kenton and L. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, Minneapolis, MN, USA, 2019, pp. 4171–4186.

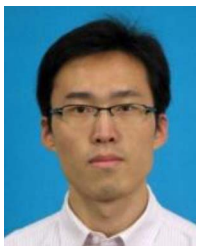
Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.

[37] T. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, Toulon, France, 2017, pp. 1–14.
[38] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Long Beach, CA, USA, 2017, Art. no. 30.
[39] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, New Orleans, LA, USA, 2019, pp. 9104–9120.
[40] T. He, Y.-S. Ong, and L. Bai, "Learning conjoint attentions for graph neural nets," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, Vancouver, British Columbia, Canada, 2021, pp. 2641–2653.
[41] M. Ahmadi, M. Safayani, and A. Mirzaei, "Deep graph clustering via mutual information maximization and mixture model," *Knowl. Inf. Syst.*, vol. 66, pp. 4549–4572, Apr. 2024, doi: 10.1007/s10115-024-02097-4.
[42] M. Hailemariam, S. Lynden, A. Matono, and T. Amagasa, "Self-attention-based data augmentation method for text classification," in *Proc. 15th Int. Conf. Mach. Learn. Comput.*, Hangzhou, Jiangsu, China, 2023, pp. 239–244.
[43] T. Gao, X. Yao, and D. Chen, "SimCSE: Simple contrastive learning of sentence embeddings," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Mexico, 2021, pp. 56952–56960.
[44] L. Yang, F. Wu, Y. Wang, J. Gu, and Y. Guo, "Masked graph convolutional network," in *Proc. Int. Joint Conf. Artif. Intell.*, Munich, Germany, 2019, pp. 4070–4077.
[45] Y. Rong, W. Huang, T. Xu, and J. Huang, "Dropedge: Towards deep graph convolutional networks on node classification," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 2689–2707.
[46] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, "Graphsaint: Graph sampling based inductive learning method," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1356–1375.
[47] K. Liu, J. He, and Y. Chen, "A topic-enhanced Dirichlet model for short text stream clustering," *Neural Comput. Appl.*, vol. 36, no. 14, pp. 8125–8140, Mar. 2024, doi: 10.1007/s00521-024-09480-w.
[48] M. Shi, Y. Zhuang, Y. Tang, M. Lin, X. Zhu, and J. Liu, "Web service network embedding based on link prediction and convolutional learning," *IEEE Trans. Serv Comput.*, vol. 15, no. 6, pp. 3620–3633, Nov./Dec. 2022, doi: 10.1109/TSC.2021.3103481.
[49] Z. Kang, Z. Lin, X. Zhu, and W. Xu, "Structured graph learning for scalable subspace clustering: From single view to multiview," *IEEE Trans. Cybern.*, vol. 52, no. 9, pp. 8976–8986, Sep. 2022, doi: 10.1109/TCYB.2021.3061660.
[50] J. Lv, Z. Kang, X. Lu, and Z. Xu, "Pseudo-supervised deep subspace clusterin," *IEEE Trans. Image Process.*, vol. 30, pp. 5252–5263, 2021, doi: 10.1109/TIP.2021.3079800.
[51] E. Pan and Z. Kang, "Beyond homophily: Reconstructing structure for graph-agnostic clustering," in *Proc. Int. Conf. Mach. Learn.*, Seattle, WA, USA, 2023, pp. 26868–26877.
[52] G. Chao, S. Sun, and J. Bi, "A survey on multiview clustering," *IEEE Trans. Artif. Intell.*, vol. 2, no. 2, pp. 146–168, Apr. 2021, doi: 10.1109/TAI.2021.3065894.
[53] X. Yu, Y. Jiang, G. Chao, and D. Chu, "Deep contrastive multi-view subspace clustering with representation and cluster interactive learning," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 1, pp. 188–199, Jan. 2025, doi: 10.1109/TKDE.2024.3484161.
[54] G. Chao, Y. Jiang, and D. Chu, "Incomplete contrastive multi-view clustering with high-confidence guiding," in *Proc. AAAI Conf. Artif. Intell.*, Vancouver, British Columbia, Canada, 2024, vol. 38, no. 10, pp. 11221–11229. [Online]. Available: https://doi.org/10.1609/aaai.v38i10.29000
[55] D. Nielsen, K. Enevoldsen, and P. Schneider-Kamp, "Encoder vs decoder: Comparative analysis of encoder and decoder language models on multilingual NLU tasks," 2024, *arXiv:2406.13469*.
[56] J. Kocoń et al., "ChatGPT: Jack of all trades, master of none," *Inf. Fusion*, vol. 99, Nov. 2023, Art. no. 101861, doi: 10.1016/j.inffus.2023.101861.

Qiang Hu received the PhD degree from the Shandong University of Science and Technology, Qingdao, China, in 2014. He is currently a professor with the College of Information Science and Technology, Qingdao University of Science and Technology, Qingdao, China. He has taken in 8 projects supported by the National Nature Science Foundation, the National Key Basic Research Developing Program, and other projects at provincial levels. He has published more than 20 papers in the international academic publications. His research interests include in formal method, service computing, and text mining.

Haoquan Qi is currently working toward the doctor's degree with the School of Computer Science and Technology, Donghua University, Shanghai, China. His research directions are services computing and artificial intelligence. He has conducted research work on Mashup service clustering and recommendation, published papers and applied for patents.

Shengzhi Du received the PhD degree from Nankai University, in 2005. He was appointed as permanent professor with the Department of Electrical and Mining Engineering, South Africa University. From 2014, he was appointed as permanent professor with the Department of Mechanical Engineering then Department of Electrical Engineering as program leaders. He is also adjunct professor with Nelson Mandela University. His research interests include intelligent human-machine systems, computer vision, image analysis and understanding, and artificial intelligence.



Pengwei Wang (Member, IEEE) received the PhD degree in computer science from Tongji University, Shanghai, China, in 2013. He finished his postdoctoral research work with the Department of Computer Science, University of Pisa, Italy, in 2015. Currently, he is an associate professor with the School of Computer Science and Technology, Donghua University, Shanghai, China. His research interests include cloud and edge computing, serverless, and services computing. He has published more than 100 papers on premier international journals and conferences, including *IEEE Transactions on Services Computing*, *IEEE Transactions on Computers, IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Cloud Computing*, etc.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY MADRAS. Downloaded on December 04,2025 at 07:27:34 UTC from IEEE Xplore. Restrictions apply.